

Secure Multi-Vendor E-Commerce Marketplace Platform

Production-grade Java full-stack marketplace platform built for a multi-vendor commerce use case. The system is designed around secure vendor onboarding, advanced product discovery, persistent shopping workflows, real-time order tracking, and role-based dashboards for customers, vendors, and administrators.

This project aligns with the LogicVeda Java Full Stack domain brief and is structured to reflect an industry-ready implementation using modern backend, frontend, data, messaging, observability, and deployment tooling.

Table of Contents

CONTENTS

1. Project Overview	1
2. Business Goal	1
3. Core Features	2
4. Technology Stack	3
5. Architecture Overview	4
6. Suggested Repository Structure	5
7. Key Workflows	6
8. Getting Started	7
9. API and Service Scope	8
10. Security Considerations	9
11. Observability and Reliability	9
12. Deployment Strategy	9
13. Execution Roadmap	10
14. Testing Strategy	11
15. Future Enhancements	12
16. Submission Notes	12

1. PROJECT OVERVIEW

The platform enables multiple vendors to onboard, manage products, and fulfill customer orders within a single marketplace. Customers can browse a rich catalog, search with filters and autocomplete, maintain a persistent cart and wishlist, place orders, and receive live order updates. Vendors can manage inventory and product listings, while admins oversee onboarding, approvals, analytics, and platform operations.

The broader goal is to simulate the architecture and workflows used by large-scale marketplaces such as Amazon, Flipkart, Etsy, and Meesho while staying practical for a portfolio-grade full-stack project.

2. BUSINESS GOAL

This project is intended to solve common marketplace challenges:

- secure onboarding and approval of third-party vendors
- scalable catalog and search across large product inventories
- low-latency cart and checkout experiences
- reliable order lifecycle processing with real-time visibility
- operational control through dashboards, analytics, and observability

2.1 Target outcomes

- Support high traffic through horizontally scalable services
- Deliver fast search and API performance
- Reduce cart abandonment with persistent cart flows and reminders
- Improve conversion through better discovery and checkout UX
- Maintain strong uptime, traceability, and operational resilience

3. CORE FEATURES

ID	Feature	Description
F-01	Authentication and Role Management	Secure signup/login with JWT access and refresh tokens, role-based access, optional OAuth2 and 2FA support
F-02	Vendor Onboarding	Vendor registration, simulated KYC review, approval workflow, and controlled product visibility
F-03	Product Catalog and Search	Product and variant management with category browsing, full-text search, filters, facets, and autocomplete
F-04	Cart and Wishlist	Redis-backed persistent cart, guest cart merge, wishlist storage, and reminder-ready flow
F-05	Orders and Payments	Checkout, order creation, payment simulation or integration, inventory updates, and idempotent processing
F-06	Real-Time Order Tracking	WebSocket-based updates for order status transitions and customer notifications
F-07	Admin and Vendor Dashboards	Role-specific panels for users, products, orders, payouts, analytics, and reporting
F-08	Notifications	Email and SMS notifications for order events, low-stock alerts, and promotional communication

4. TECHNOLOGY STACK

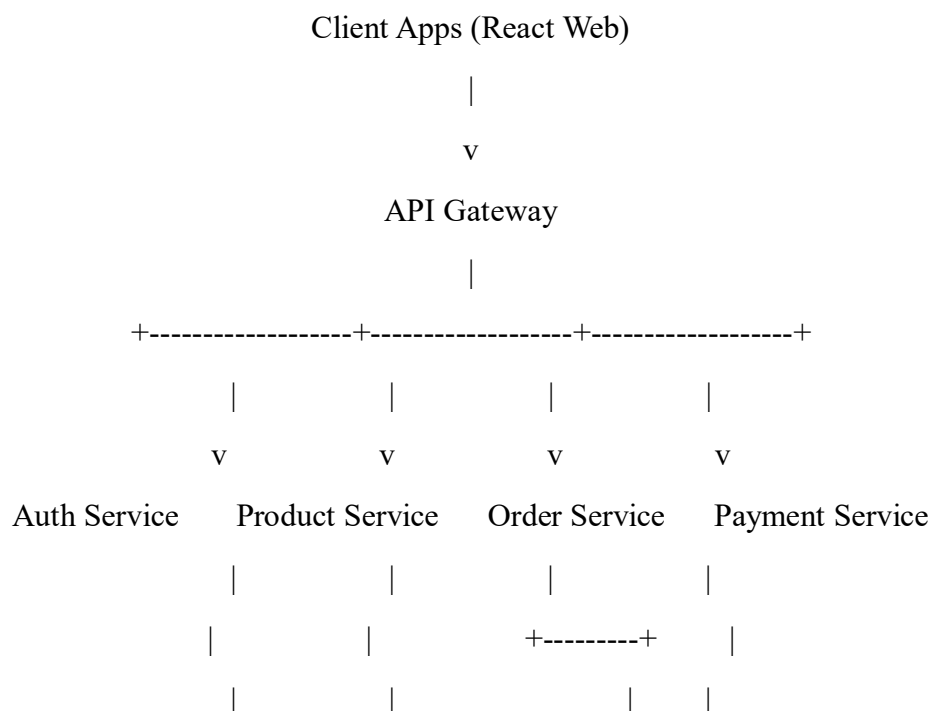
Layer	Technology
Backend	Java 21, Spring Boot 3.x, Spring Security, Spring Data JPA, Spring WebFlux
Frontend	React 19, TypeScript, Vite
UI	Tailwind CSS, shadcn/ui
State Management	TanStack Query, Zustand
Database	PostgreSQL 17
Cache	Redis 7
Search	Elasticsearch 8
Messaging	Kafka
Auth	JWT, OAuth2
Payments	Stripe simulation or integration
Notifications	SendGrid, Twilio
Containers	Docker
Orchestration	Kubernetes, Helm
CI/CD	GitHub Actions
Monitoring	Prometheus, Grafana, Loki, Sentry
Tracing	OpenTelemetry, Jaeger
Infrastructure	Terraform

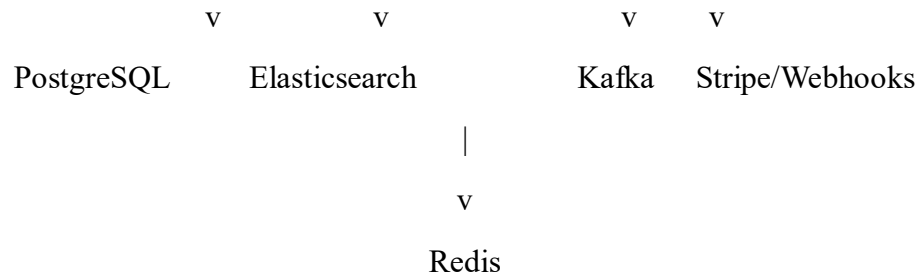
5. ARCHITECTURE OVERVIEW

The platform is designed as a modular or microservice-oriented system. A typical production setup can include:

- auth-service for authentication, authorization, token lifecycle, and user roles
- product-service for catalog, categories, variants, and inventory metadata
- order-service for checkout orchestration, order lifecycle, and status management
- payment-service for payment intent creation, webhook handling, and settlement flow
- notification-service for email, SMS, and event-driven communication
- api-gateway for unified ingress, routing, and edge concerns
- shared infrastructure for PostgreSQL, Redis, Elasticsearch, Kafka, and observability tooling

5.1 High-level flow





Notification Service consumes events and sends email/SMS updates.

Admin/Vendor dashboards consume secured APIs and analytics data.

6. SUGGESTED REPOSITORY STRUCTURE

```

|-- backend
| |-- api-gateway
| |-- auth-service
| |-- product-service
| |-- order-service
| |-- payment-service
| |-- notification-service
| `-- common
|-- frontend
| `-- web-app
|-- infra
| |-- docker
| |-- kubernetes
| |-- helm
| `-- terraform
|-- docs
| |-- architecture
| |-- api

```

```
| '-- screenshots
|-- scripts
|-- .github
| '-- workflows
'-- README.md
```

7. KEY WORKFLOWS

1. User authentication

- User signs up or logs in
- Backend issues access and refresh tokens
- Role-based permissions control access to customer, vendor, admin, and support functionality

2. Vendor onboarding

- Vendor submits registration and KYC information
- Admin reviews approval queue
- Approved vendors can publish and manage products

3. Product discovery

- Customer browses catalog by category or filters
- Search supports fuzzy matching, autocomplete, and facets
- Search index is updated when products are approved or modified

4. Cart and checkout

- Customer adds items to Redis-backed cart
- Guest cart can merge after login
- Checkout validates inventory, creates order, and triggers payment workflow

5. Order tracking

- Order status moves through payment, processing, shipping, and delivery stages
- Status changes trigger WebSocket updates and notifications

8. GETTING STARTED

The exact setup depends on how the repo is implemented. The outline below is suitable for a multi-module full-stack project.

Prerequisites

- Java 21
- Maven 3.9+
- Node.js 20+
- PostgreSQL 17
- Redis 7
- Elasticsearch 8
- Kafka
- Docker and Docker Compose

8.1 Backend setup

```
cd backend
```

```
mvn clean install
```

Run services individually or through your preferred orchestrator.

Example:

```
cd backend/auth-service
```

```
mvn spring-boot:run
```

8.2 Frontend setup

```
cd frontend/web-app
```

```
npm install
```

```
npm run dev
```

8.3 Environment configuration:

Create environment files or configuration profiles for:

database connection

Redis host and port

Elasticsearch URL

Kafka bootstrap servers

JWT secret and token expiry

Stripe keys

SendGrid and Twilio credentials

object storage or image hosting configuration

9. API AND SERVICE SCOPE

Example endpoint groups that this project can expose:

- /api/auth :for signup, login, refresh, logout, profile
- /api/vendors :for onboarding, approval, vendor profile, payouts
- /api/products :for CRUD, search, filters, variants, inventory
- /api/cart: for add, update, remove, merge, fetch cart
- /api/orders :for checkout, history, status tracking
- /api/payments: for payment intent, webhook, refund simulation
- /api/admin: for users, approvals, analytics, and platform management

10. SECURITY CONSIDERATIONS

The platform is designed with production-oriented security practices:

- JWT access and refresh token strategy
- secure password hashing with Argon2id
- RBAC for customer, vendor, admin, and support roles
- rate limiting for auth-sensitive endpoints
- input validation and global exception handling
- OWASP Top 10 aware design
- secrets managed via environment variables or vault tooling
- idempotency for payment and order creation flows
- secure cookies and HTTPS-first deployment

11. OBSERVABILITY AND RELIABILITY

To support a production-ready system, the project should include:

- application metrics through Micrometer and Prometheus
- dashboards in Grafana
- centralized logs via Loki or similar tooling
- distributed tracing with OpenTelemetry and Jaeger
- error tracking with Sentry
- health probes and readiness checks
- retries, circuit breakers, and dead-letter handling where appropriate

12. DEPLOYMENT STRATEGY

Recommended deployment progression:

1. Local development with Docker Compose

2. Containerized services with multi-stage Docker builds
3. Kubernetes manifests or Helm charts for deployment
4. CI/CD pipelines using GitHub Actions
5. Cloud deployment on AWS, GCP, or Azure

Typical production concerns include:

- autoscaling
- secrets management
- ingress routing
- persistent volumes for stateful services
- release strategies such as rolling, blue-green, or canary deployment

13. EXECUTION ROADMAP

Week 1

- bootstrap backend modules
- configure security, auth, and user roles
- set up PostgreSQL schemas and Flyway
- implement vendor onboarding basics
- add Redis-backed cart support
- establish order and payment foundations

Week 2

- bootstrap React frontend
- build auth pages and route protection
- implement catalog UI and search
- integrate Elasticsearch
- build cart and wishlist experience

Week 3

- complete checkout flow
- implement real-time order tracking
- build vendor dashboard
- build admin dashboard
- add email and SMS notifications

Week 4

- containerize services
- create Kubernetes deployment assets
- add CI/CD
- deploy to cloud
- enable monitoring and tracing
- run QA, load testing, and documentation polish

14. TESTING STRATEGY

Recommended coverage for a project of this scope:

- unit tests for service logic and utilities
- integration tests for auth, catalog, cart, and order flows
- repository tests for persistence behavior
- API tests with Postman or REST Assured
- frontend component and page tests
- end-to-end tests for login, search, cart, checkout, and order tracking
- performance tests for high-traffic scenarios
- security scans for common vulnerabilities

15. FUTURE ENHANCEMENTS

- recommendation engine using behavioral data
- product reviews and rating moderation
- coupon and promotion engine
- seller commission and settlement reports
- multi-region deployment support
- image optimization and CDN integration
- AI-assisted search, support, or merchandising

16. SUBMISSION NOTES

This README is written to satisfy the requirement for a detailed, professional project README in the repository. For final submission quality, pair it with:

- a public live demo URL
- a short demo video
- architecture screenshots
- sample credentials if needed for evaluation
- API documentation or Postman collection
- deployment notes and screenshots

16.1 Status

This repository README currently documents the intended platform architecture and scope based on the project brief. As implementation files are added, this document should be updated with:

- actual module names

- real setup commands
- live URLs
- screenshots
- API examples
- test coverage details
- known limitations and completed milestones

16.2 License

This project is intended for educational, internship, and portfolio use unless a separate license is added to the repository.